КУРС: МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ

Метод решающих деревьев

Команда №9

Андреев Ярослав, Тимошенко Дарья

ВВЕДЕНИЕ

Что такое решающие деревья?

Решающее дерево (Decision Tree) — это непараметрический метод обучения с учителем, используемый как для классификации, так и для регрессии. Модель представляет собой иерархическую древовидную структуру решающих правил вида «ЕСЛИ признак X ≤ порога, ТО...».

- **Непараметрическая природа:** Не требует предположений о нормальности распределения данных или линейности связей.
- **Интерпретируемость (White-Box):** Логика принятия решений прозрачна и может быть напрямую представлена в виде правил.
- **Универсальность:** Естественным образом работает с числовыми и категориальными признаками, решает задачи мультиклассификации.
- **Фундамент современных ансамблей:** Является базовым строительным блоком для Random Forest, Gradient Boosting (XGBoost, CatBoost).

§1

Постановка задачи

Какая математическая и прикладная проблема решается?

ПОСТАНОВКА ЗАДАЧИ

Что дано? Формализация задачи

Пусть задана обучающая выборка из N независимых наблюдений:

$$X = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}, \quad \text{где } x_i \in R^M, \; y_i \in Y$$

- **Пространство признаков:** $x_i = (x_{i1}, x_{i2}, \ldots, x_{iM})$ — вектор числовых или категориальных характеристик объекта.
- **Целевое пространство Y:** Для бинарной классификации: $Y = \{0, 1\}$; для мультиклассовой: $Y = \{1, 2, \ldots, K\}$; для регрессии: $Y \subseteq R$.
- **Целевая функция f(x):** Необходимо построить алгоритм $a(x): R^M \to Y$, минимизирующий эмпирический риск и ошибку обобщения:
- **Функция потерь L(y, a(x)):** $Q(a, X) = (1 / N) * \Sigma L(y_i, a(x_i)) \to \min$

ПОСТАНОВКА ЗАДАЧИ

В чём проблема? Ограничения классических методов

При решении реальных задач аналитики сталкиваются с фундаментальными трудностями:

- **Нелинейность разделяющих поверхностей:** Линейные классификаторы (логистическая регрессия, линейный SVM) бессильны без сложного ручного конструирования признаков (Feature Engineering).
- **Проблема «черного ящика» (Black Box):** Нейронные сети и ядерные методы не дают ответа на вопрос, ПОЧЕМУ принято решение, что недопустимо в медицине и банковском регулировании.
- **Разнородность признаков (Heterogeneous Data):** Данные часто содержат смесь чисел разного масштаба, категорий, пропусков и выбросов. Линейные модели и kNN требуют тщательной нормализации.
- **Вычислительная сложность поиска оптимального дерева:** Поиск глобально оптимального дерева решений доказанно является NP-полной задачей.

ПОСТАНОВКА ЗАДАЧИ

Что требуется от метода?

Разработать эффективный эвристический алгоритм, который:

1. Строит кусочно-постоянные разделяющие поверхности: Способен аппроксимировать произвольные нелинейные зависимости.

2. Обеспечивает полную интерпретируемость: Каждый лист соответствует цепочке логических правил ЕСЛИ-ТО.

3. Инвариантен к монотонным преобразованиям: Не требует нормализации признаков, масштабирования или центрирования.

4. Автоматически отбирает информативные признаки: Игнорирует шумовые переменные в процессе жадного расщепления.

5. Обучается за полиномиальное время: Использует жадные локально-оптимальные эвристики на каждом шаге ветвления.

Именно эти требования идеально выполняет метод решающих деревьев (CART / ID3).

§2

Области применения

Где и как решающие деревья используются на практике

ОБЛАСТИ ПРИМЕНЕНИЯ

В машинном обучении и Data Science

- **Базовый алгоритм классификации и регрессии:** Используется для быстрого прототипирования (baseline) благодаря нулевым требованиям к предобработке признаков.
- **Фундамент ансамблевых методов (Ensemble Learning):** Является базовым алгоритмом для Bagging (Random Forest) и градиентного бустинга (GBDT, XGBoost, LightGBM, CatBoost).
- **Оценка важности признаков (Feature Importance):** Вычисление суммарного уменьшения критерия неоднородности (MDI / Gini Importance) для отбора ключевых факторов.
- **Заполнение пропущенных значений:** Алгоритмы суррогатных сплитов (Surrogate Splits) позволяют оценивать пропуски в данных на лету.
- **Сегментация и кластерный анализ:** Листья дерева естественным образом разбивают выборку на однородные сегменты.

ОБЛАСТИ ПРИМЕНЕНИЯ

В прикладных задачах и индустрии

- **Банковский скоринг и кредитование:** Требование регулятора (ЦБ РФ, Базель) объяснять причину отказа клиенту (например, «Стаж < 1 года И Доход < 40 000 руб.»).
- **Медицинская диагностика и клинические протоколы:** Врачи используют деревья в виде блок-схем для постановки диагнозов и назначения лекарств без права на ошибку «черного ящика».
- **Антифрод (Fraud Detection) в финтехе:** Быстрая классификация транзакций в реальном времени с минимальной задержкой инференса (микросекунды).
- **Управление оттоком клиентов (Churn Prediction):** Маркетинговые команды получают четкие профили клиентов, склонных к уходу, для персонализированных акций.
- **Промышленность и предиктивное обслуживание:** Определение условий поломки оборудования по показаниям датчиков температуры, давления и вибрации.

§3

Основная идея

Иерархическое разбиение пространства признаков

ОСНОВНАЯ ИДЕЯ

Концепция и анатомия дерева

Термин	Смысл в алгоритме
Корень (Root node)	Исходная выборка; первое решающее правило.
Внутренний узел (Internal node)	Проверка предиката: $x_j \le t$ или $x_j \in C$.
Ветка (Branch)	Результат проверки условия (Да / Нет).
Лист (Leaf node)	Терминальный узел: финальный класс или значение.
Разбиение (Split)	Разрезание области гиперплоскостью $x_j = t$.

Принцип «Разделяй и властвуй»

- Пространство признаков рекурсивно разбивается на ортогональные гиперпрямоугольники (box regions).
- Каждое ветвление — это предикат вида $[X_j \le t]$, параллельный координатным осям.
- В каждом листе предсказывается константа: мода классов (для классификации) или среднее значение (для регрессии).
- Аналогия: игра «20 вопросов», где каждый вопрос максимально уменьшает неопределенность.

§4

Математический аппарат

Критерии информативности, сплиты и оптимизация

МАТЕМАТИЧЕСКИЙ АППАРАТ

1. Критерии неоднородности (Impurity)

Пусть p_k — доля объектов класса k в узле S: $p_k = (1 / |S|) * \Sigma [y_i = k]$

- **Энтропия Шеннона (Shannon Entropy):**

$H(S) = - \Sigma_{k=1}^K p_k * \log_2(p_k)$

Измеряет меру хаоса в битах. Для чистого узла H = 0; максимум при равных долях классов (H = 1 для K=2).

- **Критерий Джини (Gini Impurity):**

$Gini(S) = 1 - \Sigma_{k=1}^K (p_k)^2 = \Sigma_{k \neq j} p_k * p_j$

Вероятность ошибочной классификации случайного объекта при случайном угадывании класса. Варьируется от 0 до 0.5 (для K=2). Вычисляется быстрее энтропии (нет логарифмов).

- **Дисперсия (для задач регрессии):**

$MSE(S) = (1 / |S|) * \Sigma_{i \in S} (y_i - y_{mean})^2$

Критерий минимизации среднеквадратичной ошибки вокруг среднего прогноза в листе.

МАТЕМАТИЧЕСКИЙ АППАРАТ

2. Прирост информации (Information Gain)

При разбиении выборки S на две части S_left и S_right по предикату [x_j ≤ t]:

$$IG(S, j, t) = Impurity(S) - (|S_left| / |S| * Impurity(S_left) + |S_right| / |S| * Impurity(S_right))$$

- **Смысл формулы:** Разность между неопределенностью родительского узла и взвешенной суммой неопределенностей дочерних узлов.
- **Максимизация IG:** Алгоритм выбирает пару (j*, t*), дающую максимальное очищение дочерних узлов от хаоса: (j*, t*) = argmax IG(S, j, t).
- **Проблема чистого Information Gain:** Критерий смещен в сторону признаков с большим числом уникальных значений (например, ID клиента).
- **Модификация Gain Ratio (C4.5):** GainRatio(S, A) = IG(S, A) / SplitInfo(S, A), где SplitInfo(S, A) = - Σ (|S_v| / |S|) * log_2(|S_v| / |S|). Нормализует прирост на энтропию самого разбиения.

МАТЕМАТИЧЕСКИЙ АППАРАТ

3. Алгоритм поиска наилучшего сплита

Шаг 1. Перебор признаков: Итерируемся по всем доступным признакам $j \in \{1, 2, ..., M\}$.

Шаг 2. Сортировка значений: Для выбранного признака сортируем уникальные значения выборки: $x_{(1)j} < x_{(2)j} < ... < x_{(U)j}$.

Шаг 3. Генерация порогов-кандидатов: В качестве порогов t_k берутся средние точки между соседними значениями: $t_k = (x_{(k)j} + x_{(k+1)j}) / 2$. Всего не более U - 1 порогов.

Шаг 4. Оценка качества сплита: Для каждого порога делим выборку и вычисляем $IG(S, j, t_k)$ через Gini или Entropy.

Шаг 5. Оптимизационный выбор: Фиксируем пару $(j^*, t^*) = \text{argmax}_{j, t} IG(S, j, t)$. Если максимальный $IG \leq 0$ или узел чистый — деление прекращается.

МАТЕМАТИЧЕСКИЙ АППАРАТ

4. Семейство алгоритмов: ID3, C4.5, CART

Характеристика	ID3 (1986, Quinlan)	C4.5 (1993, Quinlan)	CART (1984, Breiman)
Критерий расщепления	Information Gain (Энтропия)	Gain Ratio (Нормированный IG)	Gini Impurity / Дисперсия
Тип ветвления	Многозначное (по всем категориям)	Многозначное и бинарное	Строго бинарное (два потомка)
Типы признаков	Только категориальные	Категориальные и числовые	Категориальные и числовые
Тип задачи	Классификация	Классификация	Классификация и Регрессия
Работа с пропусками / Стрижка	Нет / Нет стрижки	Вероятности / Post-pruning	Суррогатные сплиты / Cost-Complexity

МАТЕМАТИЧЕСКИЙ АППАРАТ

5. Критерии останова и регуляризация (Pre-pruning)

Без ограничений дерево будет расти до 100% чистых листьев, что неизбежно ведет к катастрофическому переобучению (Overfitting). Ранняя остановка (Pre-pruning) регулируется параметрами:

- **max_depth (Максимальная глубина):** Ограничивает длину пути от корня к листу. Защищает от слишком мелких локальных разбиений.
- **min_samples_split (Мин. объектов для расщепления):** Узел не делится, если количество объектов в нем меньше заданного порога (например, < 5 или < 10).
- **min_samples_leaf (Мин. объектов в листе):** Оба дочерних узла обязаны содержать не менее заданного числа примеров; отсекает листья для единичных выбросов.
- **min_impurity_decrease (Мин. прирост качества):** Разбиение выполняется только если достигнутый Information Gain превышает заданный порог $\varepsilon > 0$.
- **max_leaf_nodes (Макс. число листьев):** Ограничивает общее число терминальных областей в пространстве признаков.

МАТЕМАТИЧЕСКИЙ АППАРАТ

6. Стрижка деревьев (Cost-Complexity Pruning)

Post-pruning: Сначала строим полное дерево T_max, затем удаляем малоэффективные ветви снизу вверх.

$$R_\alpha(T) = R(T) + \alpha * |T|$$

- **R(T) — ошибка дерева на выборке:** Суммарная ошибка классификации или сумма квадратов остатков.
- **|T| — сложность дерева:** Количество терминальных листьев в дереве.
- **α ≥ 0 — параметр регуляризации (штраф за сложность):** При α = 0 оптимально полное дерево T_max; при α → ∞ оптимален единственный корневой лист.
- **Механизм выбора оптимального α:** С помощью кросс-валидации (k-fold CV) находят значение α, минимизирующее ошибку на валидационной выборке. Это гарантирует максимальную обобщающую способность.

§5

Алгоритм

Пошаговый процесс построения дерева решений (CART)

АЛГОРИТМ

Общая схема построения дерева (CART)

1. Инициализация: Поместить всю обучающую выборку S_0 в корневой узел дерева. Установить глубину depth = 0.

2. Проверка условий останова: Если глубина ≥ max_depth, или $|S|$ < min_samples_split, или все объекты одного класса — создать лист с преобладающим классом и завершить ветвь.

3. Поиск наилучшего сплита: Перебрать все признаки j и допустимые пороги t. Найти разбиение, максимизирующее Information Gain (минимизирующее Gini/MSE).

4. Проверка информативности: Если лучший прирост информации IG ≤ min_impurity_decrease — прекратить деление и объявить узел листом.

5. Разбиение выборки: Разделить текущее подмножество на две непересекающиеся части: $S_{left} = \{x: x_j \le t\}$ и $S_{right} = \{x: x_j > t\}$.

6. Рекурсивный вызов: Повторить шаги 2–5 независимо для левого и правого дочерних узлов с увеличением глубины: depth + 1.

7. Завершение построения: Сформировать итоговую древовидную структуру графа и сохранить решающие правила.

АЛГОРИТМ

Сложность и рекомендуемые параметры

Параметр	Типичный диапазон	Влияние на модель
max_depth	3 – 10	Меньше — выше смещение (underfit), больше — переобучение.
min_samples_split	2 – 20	Предотвращает деление слишком малых групп.
min_samples_leaf	1 – 10	Сглаживает прогноз, подавляет выбросы в листьях.
criterion	gini, entropy	Gini быстрее в вычислениях; Entropy дает похожий результат.

Вычислительная сложность

- **Обучение (Training):**
O(M ∗ N * log N * depth)
где N — число объектов, M — число признаков. Сортировка каждого признака занимает O(N log N).

- **Инференс (Prediction):**
O(depth) ≤ O(log N)
Один из самых быстрых инференсов в ML! Требует лишь цепочки сравнений чисел.

- **Память (Memory):**
O(node_count) — компактное представление в виде графа узлов.

§6

Численный пример

Пошаговый ручной расчет дерева для кредитного скоринга

ЧИСЛЕННЫЙ ПРИМЕР

1. Постановка задачи и обучающая выборка

№ Клиента	Доход, тыс. руб.	Кредитная история	Кредит (Класс Y)
1	25	Плохая (0)	0 (Отказ)
2	35	Плохая (0)	0 (Отказ)
3	45	Хорошая (1)	0 (Отказ)
4	55	Плохая (0)	0 (Отказ)
5	65	Хорошая (1)	1 (Одобрен)
6	75	Хорошая (1)	1 (Одобрен)
7	85	Хорошая (1)	1 (Одобрен)
8	95	Хорошая (1)	1 (Одобрен)

Параметры задачи:

- Объем выборки: N = 8 заявителей.
- Признак X_1: Доход клиента (числовой, от 25 до 95 тыс. руб.).
- Признак X_2: Кредитная история (бинарный: 0 — Плохая, 1 — Хорошая).
- Целевая переменная Y: 0 — Отказ, 1 — Кредит одобрен.
- Баланс классов: ровно 4 отказа (y=0) и 4 одобрения (y=1).
- Цель: построить дерево решений CART с критериями Энтропии и Джини.

ЧИСЛЕННЫЙ ПРИМЕР

2. Расчет базовой неопределенности корня S_0

В корневом узле S_0 находятся все N = 8 объектов: 4 класса '0' и 4 класса '1'.

Вероятности классов: p_0 = 4 / 8 = 0.5, p_1 = 4 / 8 = 0.5

- **Расчет энтропии корня H(S_0):**

H(S_0) = - (p_0 * log_2(p_0) + p_1 * log_2(p_1))
H(S_0) = - (0.5 * log_2(0.5) + 0.5 * log_2(0.5)) = - (0.5 * (-1) + 0.5 * (-1)) = 1.0000 бит
Максимальный возможный хаос для двух классов! Неопределенность максимальна.

- **Расчет критерия Джини корня Gini(S_0):**

Gini(S_0) = 1 - ((p_0)^2 + (p_1)^2) = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5000
Максимальная неоднородность по критерию Джини для двух классов.

ЧИСЛЕННЫЙ ПРИМЕР

3. Кандидат №1: Расщепление по признаку «Доход»

Проверяем порог t = 50 тыс. руб. (предикат: [Доход ≤ 50]):

- **Левый узел S_L (Доход ≤ 50 тыс.):**

Объекты {1, 2, 3} (N_L = 3). Все имеют класс Y = 0 (3 отказа, 0 одобрений).
p_0 = 3/3 = 1.0, p_1 = 0/3 = 0.0
H(S_L) = - (1.0 * log_2(1.0) + 0) = 0.0000 бит (Абсолютно чистый узел!)
Gini(S_L) = 1 - (1.0^2 + 0^2) = 0.0000

- **Правый узел S_R (Доход > 50 тыс.):**

Объекты {4, 5, 6, 7, 8} (N_R = 5). Содержит 1 отказ (объект 4) и 4 одобрения (5, 6, 7, 8).
p_0 = 1/5 = 0.2, p_1 = 4/5 = 0.8
H(S_R) = - (0.2 * log_2(0.2) + 0.8 * log_2(0.8)) = - (0.2 * (-2.3219) + 0.8 * (-0.3219)) = 0.7219 бит
Gini(S_R) = 1 - (0.2^2 + 0.8^2) = 1 - 0.68 = 0.3200

- **Взвешенная энтропия и Information Gain:**

H_split = (3 / 8) * H(S_L) + (5 / 8) * H(S_R) = (3/8) * 0 + (5/8) * 0.7219 = 0.4512 бит
IG(S_0, Доход ≤ 50) = H(S_0) - H_split = 1.0000 - 0.4512 = 0.5488 бит (Δ Gini = 0.3000)

ЧИСЛЕННЫЙ ПРИМЕР

4. Кандидат №2 и выбор лучшего расщепления

Проверяем предикат по второму признаку: [Кредитная история == 'Плохая']:

- **Распределение объектов по истории:**

Левый узел (Плохая): {1, 2, 4} (N_L = 3) → все Y = 0 (3 отказа, H = 0.0000 бит).
Правый узел (Хорошая): {3, 5, 6, 7, 8} (N_R = 5) → 1 отказ (объект 3) и 4 одобрения (5, 6, 7, 8) → H = 0.7219 бит.

- **Показатели информативности:**

H_split = (3/8) * 0 + (5/8) * 0.7219 = 0.4512 бит → IG = 0.5488 бит.
Кандидаты дают одинаковый прирост IG! Выбираем «Доход ≤ 50» как непрерывный признак, позволяющий лучше разделить пространство.

ИТОГ ШАГА 1: Создан корень с предикатом [Доход ≤ 50]. Левая ветка ({1, 2, 3}) становится терминальным Листом 1 со 100% классом 'Отказ' (y = 0). Правая ветка требует дальнейшего деления.

ЧИСЛЕННЫЙ ПРИМЕР

5. Рекурсивное расщепление узла S_R

В узле S_R (Доход > 50) осталось 5 объектов: {4, 5, 6, 7, 8}. Текущая энтропия H(S_R) = 0.7219 бит.

- **Проверка признака «Кредитная история» на подвыборке S_R:**
Объект 4: Доход 55, Кредитная история = 0 (Плохая), Класс Y = 0 (Отказ)
Объекты 5, 6, 7, 8: Кредитная история = 1 (Хорошая), Класс Y = 1 (Одобрен)

- **Разбиение по предикату [Кредитная история == 'Плохая']:**
Левый потомок S_RL: только объект {4} (N = 1). Класс Y = 0 → H(S_RL) = 0.0000 бит (Чистый лист!)
Правый потомок S_RR: объекты {5, 6, 7, 8} (N = 4). Все класс Y = 1 → H(S_RR) = 0.0000 бит (Чистый лист!)

- **Расчет прироста информации:**
H_split = (1 / 5) * 0 + (4 / 5) * 0 = 0.0000 бит!
IG(S_R, История) = 0.7219 - 0.0000 = 0.7219 бит (Максимально возможное очищение!)

ЧИСЛЕННЫЙ ПРИМЕР

6. Итоговая структура дерева и правила

Полученное дерево решений (глубина = 2):

[КОРЕНЬ] Доход ≤ 50 тыс. руб.?
├── ДА: ──► [ЛИСТ 1] Отказ (Y = 0) [N=3]
└── НЕТ: ──► [УЗЕЛ 2] Кредитная история == Плохая?
├── ДА: ──► [ЛИСТ 2] Отказ (Y = 0) [N=1]
└── НЕТ: ──► [ЛИСТ 3] Одобрен (Y = 1) [N=4]

Точность на обучающей выборке: 100% (8 из 8 объектов классифицированы безошибочно).

Инференс для нового клиента:

- **Новый клиент:** Доход = 70 тыс. руб., Кредитная история = 'Хорошая'.
- **Проверка корня:** Доход (70) ≤ 50? → НЕТ. Переход в правый узел 2.
- **Проверка узла 2:** Кредитная история == 'Плохая'? → НЕТ. Переход в Лист 3.
- **Решение алгоритма:** Кредит ОДОБРЕН (Класс 1).
- **Скорость вывода:** Всего 2 бинарных сравнения (O(depth) = O(2)). Мгновенное принятие прозрачного решения.

§7

Пример решения на Python

Архитектура и исходный код реализации с нуля на NumPy

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Python: Структура узла Node и расчет критериев

src/decision_tree.py (Python 3.x / NumPy)

```python
class Node:
    def __init__(self, feature=None, threshold=None,
                 left=None, right=None, *, value=None):
        self.feature, self.threshold = feature, threshold
        self.left, self.right = left, right
        self.value = value

    @property
    def is_leaf(self):
        return self.value is not None

def _impurity(self, y):
    m = len(y)
    if m == 0: return 0.0
    p = np.bincount(y, minlength=self.n_classes_) / m
    if self.criterion == 'gini':
        return 1.0 - np.sum(p ** 2)
    elif self.criterion == 'entropy':
        p_pos = p[p > 0]
        return -np.sum(p_pos * np.log2(p_pos))
```

Архитектурные решения:

- **Класс Node (Узел графа):** Каждый узел хранит индекс сплит-признака feature, порог threshold, ссылки на потомков left и right, либо финальный класс value в случае листа.
- **Свойство is_leaf:** Простой логический флаг: узел считается терминальным листом, если его value определен.
- **Векторизованный метод _impurity:** Использует np.bincount для мгновенного подсчета частот классов в выборке без использования медленных циклов Python.
- **Поддержка двух критериев:** Gini Impurity (1 - Σ p_i^2) и Shannon Entropy (-Σ p_i log_2 p_i) с безопасной фильтрацией нулевых долей p > 0.

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Python: Поиск наилучшего сплита (_best_split)

src/decision_tree.py (Python 3.x / NumPy)

```python
def _best_split(self, X, y):
    m, n = X.shape
    if m < self.min_samples_split: return None, None
    current_impurity = self._impurity(y)
    best_gain, best_feat, best_thresh = -1.0, None, None

    for feat_idx in range(n):
        vals = X[:, feat_idx]
        threshs = np.unique(vals)
        if len(threshs) <= 1: continue
        threshs = (threshs[:-1] + threshs[1:]) / 2.0

        for thresh in threshs:
            left_mask = vals <= thresh
            n_l, n_r = np.sum(left_mask), np.sum(~left_mask)
            if n_l == 0 or n_r == 0: continue
            imp_l = self._impurity(y[left_mask])
            imp_r = self._impurity(y[~left_mask])
            child_imp = (n_l/m)*imp_l + (n_r/m)*imp_r
            gain = current_impurity - child_imp
            if gain > best_gain:
                best_gain, best_feat, best_thresh = gain, feat_idx, thresh

    return best_feat, best_thresh
```

Ключевые механизмы сплита:

- **Генерация непрерывных порогов:** np.unique находит уникальные значения признака, а полусумма (t[:-1] + t[1:]) / 2.0 генерирует середины интервалов.
- **Булевы маски NumPy:** Разделение массива значений происходит через логические маски left_mask = values <= thresh, что минимизирует копирование данных в памяти.
- **Взвешенная сумма неоднородностей:** child_imp рассчитывается с весами долей n_l/m и n_r/m в соответствии с формулой CART.
- **Максимизация Information Gain:** Алгоритм сохраняет тройку (best_gain, best_feat, best_thresh), обеспечивающую максимальное снижение хаоса.

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Python: Рекурсивное построение и инференс

src/decision_tree.py (Python 3.x / NumPy)

```
def _build_tree(self, X, y, depth=0):
    n_samples, n_labels = len(y), len(np.unique(y))
    if depth >= self.max_depth or n_labels == 1        or n_samples < self.min_samples_split:
        return Node(value=np.argmax(np.bincount(y, minlength=self.n_classes_)))

    feat, thresh = self._best_split(X, y)
    if feat is None:
        return Node(value=np.argmax(np.bincount(y, minlength=self.n_classes_)))

    left_mask = X[:, feat] <= thresh
    left = self._build_tree(X[left_mask], y[left_mask], depth + 1)
    right = self._build_tree(X[~left_mask], y[~left_mask], depth + 1)
    return Node(feature=feat, threshold=thresh, left=left, right=right)

def _traverse_tree(self, x, node):
    if node.is_leaf: return node.value
    if x[node.feature] <= node.threshold:
        return self._traverse_tree(x, node.left)
    return self._traverse_tree(x, node.right)
```

Рекурсия и применение:

- **Базовый случай останова:** Лист формируется при: 1) достижении max_depth; 2) чистоте узла (n_labels == 1); 3) нехватке объектов (< min_samples_split).
- **Определение ответа в листе:** np.argmax(np.bincount(y)) возвращает самый частый класс среди объектов, дошедших до данного листа.
- **Рекурсивный спуск _traverse_tree:** Для классификации нового объекта выполняется бинарный спуск по предикатам от корня к листу.
- **Асимптотическая скорость:** Время классификации строго ограничено высотой дерева O(depth), что дает микросекундный инференс на один запрос.

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Python: Результаты работы и анализ переобучения

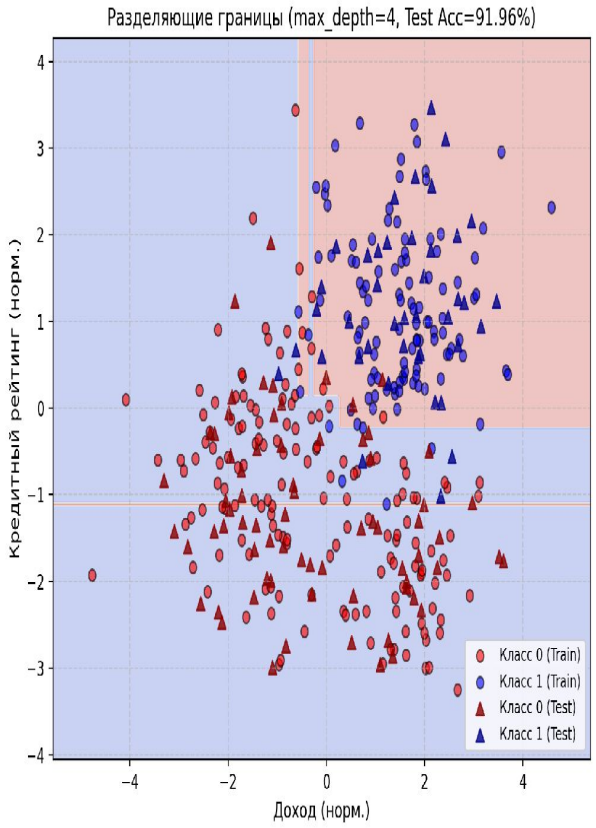
Параметры эксперимента и метрики:

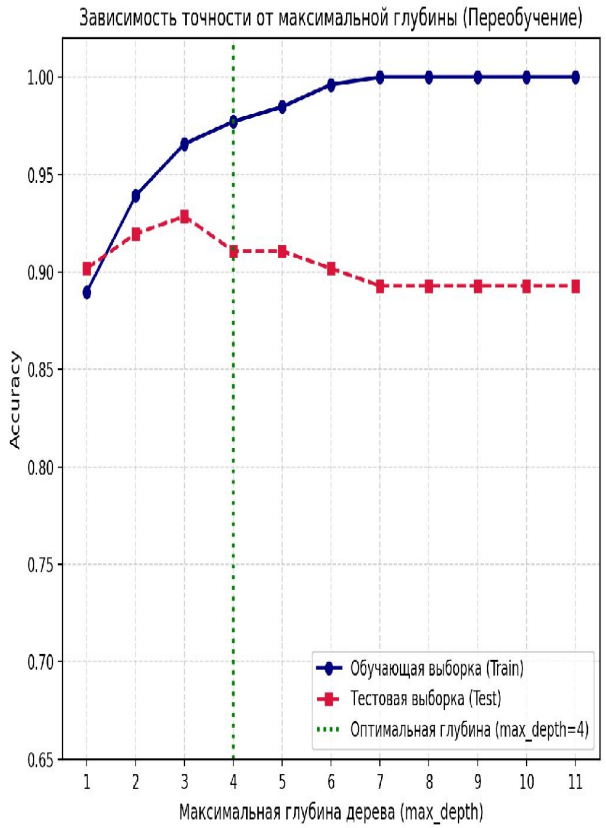
• **Синтетический датасет:** Двумерная нелинейная задача кредитного скоринга с шумом (N=375 объектов, разбиение Train 70% / Test 30%).

• **Оптимальные гиперпараметры:** max_depth = 4, min_samples_split = 5, criterion = 'gini'.

• **Метрики качества на тесте:** Accuracy Train: 97.34%, Test: 91.96% | Precision: 90%, Recall: 87.8%, F1: 88.9%

• **Вывод по переобучению:** На графике справа наглядно видно, что при max_depth > 4 точность на тесте деградирует (overfitting), подтверждая важность контроля глубины.





§8

Альтернативы и сравнение

Сравнение решающих деревьев с другими методами машинного обучения

АЛЬТЕРНАТИВЫ И СРАВНЕНИЕ

Достоинства и недостатки решающих деревьев

Ключевые преимущества (Плюсы)

✓ **Интерпретируемость (White Box):** Логика принятия решений понятна человеку и легко конвертируется в SQL/Python код (if-else).

✓ **Минимум предобработки:** Не требуется масштабирование, нормализация признаков, центрирование.

✓ **Смешанные типы признаков:** Нативно работает с числовыми и категориальными данными.

✓ **Быстрый инференс O(depth):** Требует логарифмического времени от числа объектов для предсказания.

✓ **Устойчивость к выбросам:** Пороговое деление малочувствительно к экстремальным значениям на хвостах распределения.

Ключевые недостатки (Минусы)

× **Склонность к переобучению (Overfitting):** Глубокие деревья запоминают шум выборки, теряя обобщающую способность.

× **Неустойчивость (Высокая дисперсия / Variance):** Небольшое изменение в данных может полностью перестроить структуру дерева.

× **Ортогональные границы:** Разделяющие поверхности строго параллельны осям. Диагональные границы приближаются неэффективной «лесенкой».

× **Жадный поиск (Локальный оптимум):** Алгоритм не пересматривает ранее сделанные разбиения, пропуская глобально оптимальные деревья.

× **Сложность в экстраполяции:** В задачах регрессии дерево не способно предсказывать значения за пределами обучающего диапазона.

АЛЬТЕРНАТИВЫ И СРАВНЕНИЕ

Сравнительная таблица методов машинного обучения

Метод / Алгоритм	Интерпретируемость	Сложность границы	Предобработка	Склонность к переобучению
Решающее дерево (CART)	Очень высокая (White Box)	Нелинейная (ортогональная)	Минимальная (нет норм.)	Высокая (без стрижки)
Логистическая регрессия	Высокая (веса w_i)	Строго линейная	Требует нормализации	Низкая (L1/L2 регуляризация)
k-Ближайших соседей (kNN)	Средняя (по аналогии)	Нелинейная локальная	Критична нормализация	Средняя (зависит от k)
Метод опорных векторов (SVM)	Низкая (ядерные веса)	Произвольная (RBF-ядро)	Критична нормализация	Средняя (параметры C, γ)
Случайный лес (Random Forest)	Средняя (Feature Import.)	Сложная нелинейная	Минимальная	Очень низкая (бэггинг усредняет)
Градиентный бустинг (GBDT)	Средняя (SHAP / MDI)	Максимальная точность	Минимальная	Средняя (требует подбора)

АЛЬТЕРНАТИВЫ И СРАВНЕНИЕ

Когда выбирать решающие деревья?

РЕКОМЕНДУЕТСЯ использовать, если:

• Требуется 100% объяснимость каждого предсказания (юриспруденция, кредиты, медицина).

• Данные табличные (Tabular Data) и содержат разнородные признаки (категории, числа, пропуски).

• Необходимо быстро построить базовую модель (baseline) без долгой настройки пайплайна.

• Критична ультранизкая задержка инференса (микросекунды на CPU в реальном времени).

• Нужно автоматически выявить ключевые нелинейные взаимодействия признаков.

НЕ РЕКОМЕНДУЕТСЯ использовать, если:

• Данные неструктурированные: изображения, аудио, сырой текст (здесь доминируют нейросети CNN / Transformer).

• Задача требует гладкой непрерывной экстраполяции функции за пределы обучающей выборки.

• Зависимость в данных строго линейная (линейная регрессия сработает точнее и быстрее).

• Критична максимальная точность на соревнованиях Kaggle (следует сразу переходить к ансамблям — CatBoost/XGBoost).

• Малый объем данных при высокой зашумленности (одиночное дерево переобучится).

АЛЬТЕРНАТИВЫ И СРАВНЕНИЕ

Развитие метода: Ансамбли на основе деревьев

Главная слабость одиночного дерева — высокая дисперсия (нестабильность). Ансамбли решают эту проблему:

- **Бэггинг и Случайный лес (Random Forest, Breiman 2001):**
Обучение B независимых глубоких деревьев на бутстрап-подвыборках со случайным подпространством признаков. Усреднение прогнозов снижает дисперсию без роста смещения: $Var(RF) \approx (1/B) * Var(Tree) + \rho * Var(Tree)$.

- **Градиентный бустинг (Gradient Boosting Decision Trees, Friedman 2001):**
Последовательное обучение неглубоких деревьев (stumps / shallow trees), где каждое следующее дерево аппроксимирует антиградиент функции потерь (ошибки предыдущих): $b_t(x) \approx - \partial L / \partial F_{t-1}(x)$.

- **Современные библиотеки бустинга (SOTA на табличных данных):**
  - XGBoost: регуляризация второго порядка (гессиан), приближенный поиск сплитов.
  - LightGBM: гистограммный поиск сплитов, построение дерева leaf-wise (по листьям).
  - CatBoost: нативная работа с категориальными признаками, Ordered Boosting без утечки таргета.

§9

Заключение

Ключевые итоги, выводы и литература

ЗАКЛЮЧЕНИЕ

Ключевые выводы

1. Решающие деревья — фундаментальный непараметрический метод обучения с учителем, формирующий интуитивно понятные правила ЕСЛИ-ТО.

2. Основные компоненты алгоритма: критерии неоднородности (Gini, Entropy, MSE), рекурсивный поиск наилучшего сплита и правила останова.

3. Алгоритм CART использует жадную эвристику на каждом шаге, решая задачу за полиномиальное время O(M * N * log N) вместо экспоненциального.

4. Главное конкурентное преимущество — нативная интерпретируемость (White Box) и отсутствие необходимости в нормализации признаков.

5. Главный риск — быстрое переобучение (High Variance), требующее обязательного контроля гиперпараметров (max_depth, min_samples_split) или стрижки (pruning).

6. Инференс модели крайне эффективен: O(depth), что позволяет внедрять деревья в высоконагруженные realtime-системы с микросекундными SLA.

7. Численный пример наглядно доказал, как за два шага деления исходная неопределенность H=1.0 полностью сводится к 0 с формированием точных правил.

8. Решающие деревья послужили фундаментом для мощнейших алгоритмов машинного обучения на табличных данных — Random Forest и градиентного бустинга.

ЗАКЛЮЧЕНИЕ

Современное состояние и перспективы развития

- **Объяснимый искусственный интеллект (Explainable AI / XAI):**
В эпоху строгих законодательных требований (EU AI Act) деревья решений и их локальные суррогаты (LIME, TreeSHAP) становятся золотым стандартом аудита сложных систем.

- **Дифференцируемые решающие деревья (Neural Decision Trees):**
Смягчение жестких предикатов с помощью сигмоид/softmax позволяет обучать деревья градиентным спуском эндогенно внутри нейросетевых архитектур.

- **Оптимальные решающие деревья (Optimal Trees via MIP):**
Современные алгоритмы смешанного целочисленного программирования (OCT) способны находить глобально оптимальные деревья небольшой глубины за секунды.

- **Косоугольные деревья (Oblique Decision Trees):**
Разбиения гиперплоскостями произвольной ориентации $w^T x \leq t$ преодолевают ограничение ортогональности к осям координат.

ЗАКЛЮЧЕНИЕ

Список источников и литературы

Основные фундаментальные источники:

• Breiman L., Friedman J., Stone C.J., Olshen R.A. Classification and Regression Trees. – Wadsworth & Brooks/Cole Advanced Books & Software, Monterey, CA, 1984. – 358 p.

• Quinlan J.R. Induction of Decision Trees // Machine Learning. – 1986. – Vol. 1, No. 1. – P. 81–106.

• Quinlan J.R. C4.5: Programs for Machine Learning. – Morgan Kaufmann Publishers, San Mateo, CA, 1993. – 302 p.

• Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. – 2nd ed. – Springer, New York, 2009. – 745 p.

• Воронцов К.В. Математические методы обучения по прецедентам (теория обучения машин): Учебное пособие. – М.: МФТИ, 2007. – 142 с.

• James G., Witten D., Hastie T., Tibshirani R. An Introduction to Statistical Learning with Applications in R. – Springer, 2013. – 426 p.

Интернет-ресурсы и документация:

• Scikit-learn Documentation: Decision Trees. URL: https://scikit-learn.org/stable/modules/tree.html

• Дьяконов А.Г. Деревья решений в машинном обучении. URL: https://dyakonov.org/2015/12/15/деревья-решений/